JavaScript Hands-On Exercises

Local Community Event Portal

1. JavaScript Basics & Setup

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Community Event Portal</title>
</head>
<body>
  <h1>Community Event Portal</h1>
  <script src="main.js"></script>
</body>
</html>
console.log("Welcome to the Community Portal");

window.onload = function () {
  alert("Page fully loaded");
};
```

2. Syntax, Data Types, and Operators

```
const eventName = "Baking Workshop";
const eventDate = "2026-08-15";
let seatsAvailable = 20;

console.log(`Event: ${eventName} on ${eventDate}, Seats left:
${seatsAvailable}`);

function registerSeat() {
  seatsAvailable--;
  console.log(`Seats left after registration: ${seatsAvailable}`);
}
```

```
registerSeat();
```

3. Conditionals, Loops, and Error Handling

```
const events = [
  { name: "Baking Workshop", date: "2026-08-15", seats: 5 },
  { name: "Music Fest", date: "2026-06-01", seats: 0 },
  { name: "Yoga Camp", date: "2026-09-10", seats: 12 }
];

const today = new Date("2026-07-25");

events.forEach(function (event) {
  const eventDate = new Date(event.date);
  if (eventDate >= today && event.seats > 0) {
    console.log(`Showing event: ${event.name}`);
  } else {
    console.log(`Hiding event: ${event.name}`);
  }
});

function registerUser(event) {
  try {
    if (event.seats <= 0) {
      throw new Error("No seats available");
    }
    event.seats--;
    console.log(`Registered for ${event.name}`);
  } catch (error) {
    console.log(`Registration failed: ${error.message}`);
  }
}

registerUser(events[1]);
```

4. Functions, Scope, Closures, Higher-Order Functions

```
const events = [];

function addEvent(name, date, seats, category) {
  events.push({ name, date, seats, category });
}

function registerUser(eventName) {
  const event = events.find(e => e.name === eventName);
  if (event && event.seats > 0) {
    event.seats--;
    return true;
  }
  return false;
}

function filterEventsByCategory(category, callback) {
  return events.filter(callback).filter(e => e.category === category);
}

function categoryCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}

const musicRegistrations = categoryCounter();
addEvent("Baking Workshop", "2026-08-15", 5, "Food");
addEvent("Music Fest", "2026-06-01", 10, "Music");

console.log(musicRegistrations());
console.log(filterEventsByCategory("Music", e => e.seats > 0));
```

5. Objects and Prototypes

```
function Event(name, date, seats) {
  this.name = name;
  this.date = date;
  this.seats = seats;
}

Event.prototype.checkAvailability = function () {
  return this.seats > 0;
};

const bakingEvent = new Event("Baking Workshop", "2026-08-15", 5);
console.log(bakingEvent.checkAvailability());

for (const [key, value] of Object.entries(bakingEvent)) {
  console.log(`${key}: ${value}`);
}
```

6. Arrays and Methods

```
let events = [
  { name: "Baking Workshop", category: "Food" },
  { name: "Music Fest", category: "Music" },
  { name: "Yoga Camp", category: "Fitness" }
];

events.push({ name: "Jazz Night", category: "Music" });

const musicEvents = events.filter(e => e.category === "Music");
console.log(musicEvents);

const displayCards = events.map(e => `Workshop on ${e.name}`);
console.log(displayCards);
```

7. DOM Manipulation

```
const eventContainer = document.querySelector("#eventList");

function renderEvent(event) {
  const card = document.createElement("div");
  card.className = "event-card";
  card.textContent = `${event.name} - ${event.date}`;
  eventContainer.appendChild(card);
}

function updateSeatDisplay(event) {
  const seatEl = document.querySelector(`#seats-${event.id}`);
  seatEl.textContent = `Seats left: ${event.seats}`;
}
```

8. Event Handling

```
document.querySelector("#registerBtn").onclick = function () {
  console.log("Register button clicked");
};

document.querySelector("#categoryFilter").onchange = function (e) {
  console.log("Filter changed to: " + e.target.value);
};

document.querySelector("#searchBox").addEventListener("keydown",
function (e) {
  console.log("Searching for: " + e.target.value);
});
```

9. Async JS, Promises, Async/Await

```
fetch("https://mockapi.example.com/events")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log("Error fetching events:", error));
async function loadEvents() {
```

```
  const spinner = document.querySelector("#spinner");
  spinner.style.display = "block";
  try {
    const response = await fetch("https://mockapi.example.com/events");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log("Error fetching events:", error);
  } finally {
    spinner.style.display = "none";
  }
}

loadEvents();
```

10. Modern JavaScript Features

```
function addEvent(name, date, seats = 10) {
  return { name, date, seats };
}

const event = { name: "Baking Workshop", date: "2026-08-15", seats:
5 };
const { name, date, seats } = event;
console.log(name, date, seats);

const originalEvents = [event];
const clonedEvents = [...originalEvents];
const filteredEvents = clonedEvents.filter(e => e.seats > 0);
console.log(filteredEvents);
```

11. Working with Forms

```
<form id="registerForm">
  <input type="text" name="fullName" placeholder="Name">
  <input type="email" name="email" placeholder="Email">
```

```
  <select name="eventName">
    <option value="Baking Workshop">Baking Workshop</option>
    <option value="Music Fest">Music Fest</option>
  </select>
  <span id="formError" class="error"></span>
  <button type="submit">Register</button>
</form>
document.querySelector("#registerForm").addEventListener("submit",
function (e) {
  e.preventDefault();

  const name = e.target.elements.fullName.value;
  const email = e.target.elements.email.value;
  const eventName = e.target.elements.eventName.value;
  const errorEl = document.querySelector("#formError");

  if (!name || !email) {
    errorEl.textContent = "Name and email are required";
    return;
  }

  errorEl.textContent = "";
  console.log("Registering:", name, email, eventName);
});
```

12. AJAX & Fetch API

```
function submitRegistration(userData) {
  setTimeout(() => {
    fetch("https://mockapi.example.com/register", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(userData)
    })
      .then(response => {
```

```
      if (response.ok) {
        console.log("Registration successful");
      } else {
        console.log("Registration failed");
      }
    })
    .catch(error => console.log("Error:", error));
 }, 1000);
}

submitRegistration({ name: "Sneha", email: "sneha@example.com",
event: "Baking Workshop" });
```

13. Debugging and Testing

```
document.querySelector("#registerForm").addEventListener("submit",
function (e) {
 e.preventDefault();
 console.log("Form submission started");

 const payload = {
  name: e.target.elements.fullName.value,
  email: e.target.elements.email.value
 };
 console.log("Payload:", payload);

 debugger;

 fetch("https://mockapi.example.com/register", {
  method: "POST",
  body: JSON.stringify(payload)
 }).then(res => console.log("Response status:", res.status));
});
```

Observation: registration was failing because email field name in HTML did not match e.target.elements reference, found using Console and Network tab request payload check.

14. jQuery and JS Frameworks

```
$('#registerBtn').click(function () {
  console.log("Register button clicked via jQuery");
});

$('.event-card').fadeOut(300, function () {
  $(this).fadeIn(300);
});
```

Moving to a framework like React gives component reusability and automatic UI updates through state management, instead of manually updating the DOM.